

레벨 2 회고 — 기술적 의사결정과 설계 경험

클렌 · 우아한테크코스 레벨 2 · 2026-06

관통한 기준 — 경계는 도메인의 책임이 정한다

레벨 1에서 도메인 단위의 책임을 정립했다면, 레벨 2는 그 책임 모델을 웹 환경 — 요청과 응답, 트랜잭션, 테스트 — 으로 확장하는 과정이었다. 객체 지향적 구조 위에 절차 지향적 흐름을 구현하되, 모든 결정의 기준은 하나였다. 기술 구조(테이블·계층·프레임워크)가 아니라 도메인과 비즈니스 로직의 책임이 경계를 정한다.

결정 1 · 트랜잭션 경계와 동시성 — 예약 취소와 대기 자동 승격

취소 시 예약만 삭제되어 "예약 없는 대기"라는 비정상 상태가 사용자에게 노출됐다. 취소와 승격은 하나의 비즈니스 결과이므로 단일 작업 단위로 묶었고, 테이블이 여러 개 바뀐다는 이유만으로는 묶지 않는다 — 경계의 기준은 DB 구조가 아니라 비즈니스 정합성이다.

스키마도 같은 기준으로 움직였다. 대기가 가변적인 예약 행을 참조하면 취소·승격마다 변경이 전파되므로 거부했고, 중복 저장하던 공통 개념(날짜·시간·테마)을 불변의 session으로 추출해 예약 1 : 세션 1 : 대기 N 구조로 정규화했다. 서비스가 대신하던 과거 시점 판별도 도메인(Slot)으로 옮겨 책임을 집중했다.

동시 취소 실험에서는 두 트랜잭션이 같은 1순위 대기를 읽고 중복 승격을 시도하는 경합을 발견했다. 각 트랜잭션의 원자성은 지켜졌다 — 격리, 즉 MVCC의 비잠금 읽기가 같은 대기를 읽도록 허용한 것이 원인이다. 수습은 스키마가 맡는다. 정규화로 선언 가능해진 **UNIQUE(session_id)**가 "한 슬롯에 예약은 하나"라는 비즈니스 불변식을 DB가 검사하는 제약으로 번역해, 경합의 패자를 제약 위반으로 롤백시킨다. 패자에게는 내부 승격 과정을 노출하지 않고 취소 실패만 알린다 — 재시도는 사용자의 판단이다. 이 경계는 고정이지 않다. 자동 승격이라는 요구사항 선택이 만든 경계이므로, 요구사항이 바뀌면 트랜잭션 경계도 함께 움직인다.

트랜잭션 경계 한 줄 — 부분 실패가 사용자에게 노출될 수 있는 비정상 상태를 남기는 작업들은 하나의 트랜잭션으로 묶는다.

결정 2 · 에러 응답 — 잘못된 요청에 합당한 응답

존재하지 않는 예약 조회와 중복 예약 시도가 500과 무의미한 본문으로 노출되고, 지난 날짜 예약은 201로 성공하고 있었다. 예측 가능한 위반을 4xx로 변환하고, 응답을 code(클라이언트 분기용 안정적 식별자)와 message(원인과 다음 행동)로 표준화했다. 원천 데이터의 검증 책임이 서버에 있기에 실패의 원인과 대응을 일원적으로 응답에 담을 수 있었다.

스스로 건 제약 — "다음 행동" 정보를 만들기 위해 추가 조회를 하지 않는다. 응답에 쓰는 정보는 요청-응답 흐름 내부로 한정하며, 가용 시간 안내는 기존 조회 기능의 책임이다. 올바른 요청에 올바른 응답이면 동작하는 API, 잘못된 요청에도 합당한 응답이면 바람직한 API다.

결정 3 · 테스트 — 보호할 책임의 분리

레벨 1식 순수 단위 테스트가 서비스 계층에서 막혔고, 원인을 "도메인 서비스가 아닌 애플리케이션 서비스"로 진단했다 — 책임의 종류가 테스트 단위를 결정한다. 보호 대상은 프레임워크가 아니라 우리가 작성한 비즈니스 규칙과 외부 동작 계약이다. 외부 의존 없이 검증 가능하면 단위, 결합 자체가 대상이면 통합·슬라이스, 사용자 흐름 보호면 최소한의 E2E로 검증한다. 테스트 더블은 호출 행위 확인이면 Mock, 쿼리·매핑 검증이면 인메모리 DB, 벤더 특화 기능이면 실제 DB를 쓴다. 단순 위임 계층과 프레임워크가 보장하는 영역은 테스트하지 않는다. 이 과정에서 테스트는 꺼려지는 의무에서 프로덕션 코드와 같은 유지보수 대상으로 바뀌었다.

학습 과정

사전 학습 → 토론 → 구현 → 피드백 사이클의 나선형 반복으로 진행했다. 규칙은 정답이 아니라 기준을 갈고닦는 보조 도구이자 팀 컨벤션 설정 도구로 다뤘고, 학습법 자체도 Feedback Loop — "고민보다 실패를" — 를 축으로 v1에서 v3까지 튜닝했다.

다음 과제

동시성 제어의 체계화 — 격리 수준·락·제약의 비용 비교와 실측. 그리고 결점의 근본 제거 이전에, 사용자 경험 정합성의 우선 개선.